OCP Maintenance AI Platform — Technical Documentation

> ***Version**: 1.0.0 **Last Updated**: February 2026 **Platform**: Containerized Full-Stack Web Application*
> ***Client**: OCP Group (Office Chérifien des Phosphates)*

Table of Contents

---

1. Overview

1.1 What is this project?

OCP Maintenance AI Platform is an enterprise maintenance management system with 4 modules, powered by artificial intelligence. It was designed for OCP Group, the world's largest phosphate exporter, to optimize maintenance operations at their Jorf Lasfar industrial complex (JFC1).

1.2 Main Objectives

Objective	Description
Predictive Maintenance	Use Weibull analysis and AI to predict equipment failures before they occur
RCM Optimization	Apply Reliability-Centered Maintenance methodology to optimize task strategies
SAP Integration	Generate work orders, task lists, and upload packages compatible with SAP
Multi-Language Support	Full interface in English, Spanish, and Arabic (RTL)
Role-Based Access Control	5 user roles with granular permission control
Real-Time Analytics	KPIs, health scores, OEE, MTBF/MTTR dashboards

1.3 The 4 Modules

```
Module 1: ASSET HIERARCHY AND CRITICALITY
  Equipment tree management, risk matrix assessment

Module 2: FMEA / FMECA AND RCM STRATEGY
  Failure mode analysis, RCM decision logic, task generation

Module 3: OPERATIONS AND PLANNING
  Work requests, backlog management, scheduling, work packages

Module 4: ANALYTICS, REPORTS AND CONTINUOUS IMPROVEMENT
  KPIs, Weibull analysis, RCA (5W2H), defect elimination, SAP export
```

---

2. System Architecture

2.1 General Diagram

2.2 Service Ports

Service	Port	Description
Frontend (Vite)	`5173`	React development server
Backend (FastAPI)	`8000`	API + Swagger Documentation at `/docs`
Nginx (Proxy)	`8080`	Unified entry point
Nginx (SSL)	`8443`	HTTPS (production)

2.3 Request Flow

```
Browser -> Nginx:8080
  |-- /api/*    -> FastAPI:8000 (Backend API)
  |-- /health   -> FastAPI:8000 (Health check)
  |-- /docs     -> FastAPI:8000 (Swagger UI)
  +-- /*        -> React:5173   (Frontend SPA)
```

---

3. Technology Stack

3.1 Backend

Technology	Version	Purpose
Python	3.11	Runtime
FastAPI	0.133.1	Web framework
Uvicorn	0.41.0	ASGI server
Gunicorn	25.1.0	Process manager (production)
SQLAlchemy	2.0.47	ORM
Pydantic	2.12.5	Data validation
python-jose	3.3.0	JWT tokens
bcrypt	4.2.1	Password hashing
Anthropic SDK	0.83.0	Claude AI integration
openpyxl	3.1.5	Excel generation
httpx	0.28.1	HTTP client

3.2 Frontend

Technology	Version	Purpose
React	19.1.0	UI framework
Vite	7.3.1	Build tool and dev server
Tailwind CSS	4.2.1	Utility-first styling
React Router DOM	7.13.1	Client-side routing
Radix UI	Latest	Accessible headless components
Recharts	3.7.0	Data visualization / charts
Lucide React	0.575.0	Icon library
XLSX	0.18.5	Client-side Excel export
file-saver	2.0.5	File download utility

3.3 Infrastructure

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-service orchestration
Nginx Alpine	Reverse proxy + static file serving
SQLite	Development database
PostgreSQL	Production database

4. Project Structure

```
ASSET-MANAGEMENT-SOFTWARE-master/
|
|-- api/                              # === BACKEND (FastAPI) ===
|   |-- main.py                       # Entry point: middleware, routers, health
|   |-- config.py                     # Configuration from .env
|   |-- schemas.py                    # 50+ Pydantic request/response models
|   |-- seed.py                       # Database seed script
|   |
|   |-- database/
|   |   |-- connection.py             # SQLAlchemy engine and sessions
|   |   +-- models.py                 # 25+ ORM models
|   |
|   |-- routers/                      # API endpoint modules
|   |   |-- admin.py                  # Seed, statistics, audit
|   |   |-- analytics.py              # Health scores, KPIs, Weibull
|   |   |-- auth.py                   # Login, registration, JWT
|   |   |-- backlog.py                # Backlog management
|   |   |-- capture.py                # Field data capture
|   |   |-- criticality.py            # Risk assessment
|   |   |-- dashboard.py              # Executive dashboard
|   |   |-- fmea.py                   # FMEA/FMECA, RCM decisions
|   |   |-- hierarchy.py              # Equipment tree CRUD
|   |   |-- planner.py                # AI planner recommendations
|   |   |-- rca.py                    # Root Cause Analysis
|   |   |-- reliability.py            # Spare parts, shutdowns, MOCs
|   |   |-- reporting.py              # Reports, exports, notifications
|   |   |-- sap.py                    # SAP integration
|   |   |-- scheduling.py             # Scheduling, GANTT
|   |   |-- tasks.py                  # Maintenance task CRUD
|   |   |-- work_packages.py          # Work package grouping
|   |   +-- work_requests.py          # Work request validation
|   |
|   |-- services/                     # Business logic layer (23 services)
|   |   |-- agent_service.py          # AI agent configuration
|   |   |-- analytics_service.py      # KPI calculations
|   |   |-- audit_service.py          # Audit logging
|   |   |-- auth_service.py           # JWT, password hashing
|   |   |-- backlog_service.py        # Backlog optimization
|   |   |-- criticality_service.py    # Risk matrix engine
|   |   |-- fmea_service.py           # FMEA + RCM logic
|   |   |-- hierarchy_service.py      # Equipment hierarchy
|   |   |-- planner_service.py        # Planning algorithms
|   |   |-- rca_service.py            # 5W2H analysis
|   |   |-- reliability_service.py    # Failure prediction
|   |   |-- reporting_service.py      # Report generation
|   |   |-- sap_service.py            # SAP data processing
|   |   |-- scheduling_service.py     # Scheduling optimization
|   |   |-- work_package_service.py   # WP grouping logic
|   |   |-- work_request_service.py   # WR processing
|   |   +-- ...                       # Additional services
|   |
|   +-- dependencies/
|       +-- auth.py                   # JWT dependency injection
|
|-- frontend/                         # === FRONTEND (React SPA) ===
|   |-- package.json                  # npm dependencies
```

```
|   |-- vite.config.js               # Vite configuration + proxy
|   |-- index.html                   # SPA entry point (favicon, meta)
|   |-- Dockerfile                   # Multi-stage build
|   |
|   +-- src/
|       |-- main.jsx                 # Route configuration (23 routes)
|       |-- App.jsx                  # Root layout (Sidebar + Header)
|       |-- api.js                   # API client (50+ endpoints)
|       |
|       |-- pages/                   # 23 page components
|       |   |-- Login.jsx            # Authentication
|       |   |-- Dashboard.jsx        # KPI overview
|       |   |-- Hierarchy.jsx        # Equipment tree
|       |   |-- Criticality.jsx      # Risk assessment
|       |   |-- FMEA.jsx             # Failure mode analysis
|       |   |-- FMECA.jsx            # FMEA with criticality
|       |   |-- Strategy.jsx         # RCM strategy selection
|       |   |-- WorkRequests.jsx     # Request management
|       |   |-- WorkPackages.jsx     # Task grouping + SAP export
|       |   |-- Backlog.jsx          # Maintenance backlog
|       |   |-- Scheduling.jsx       # Scheduling
|       |   |-- Planning.jsx         # Long-term planning
|       |   |-- Planner.jsx          # AI assistant chat
|       |   |-- FieldCapture.jsx     # Mobile data capture
|       |   |-- Analytics.jsx        # KPI dashboards
|       |   |-- ExecutiveDashboard.jsx# Executive summary
|       |   |-- Reports.jsx          # Report generation + export
|       |   |-- Reliability.jsx      # Spare parts, shutdowns
|       |   |-- RCA.jsx              # Root Cause Analysis
|       |   |-- DefectElimination.jsx # Defect tracking
|       |   |-- SAPReview.jsx        # SAP upload review
|       |   |-- Admin.jsx            # User management + settings
|       |   +-- Profile.jsx          # User profile
|       |
|       |-- components/              # Reusable UI components
|       |   |-- Sidebar.jsx          # Navigation menu
|       |   |-- Header.jsx           # Top bar + plant selector
|       |   |-- ProtectedRoute.jsx   # Role-based access guard
|       |   |-- ErrorBoundary.jsx    # Error handling
|       |   |-- Toast.jsx            # Notification system
|       |   |-- ConfirmDialog.jsx    # Confirmation dialogs
|       |   |-- Shared.jsx           # StatusBadge, LoadingSpinner
|       |   +-- ui/                  # 15+ Radix UI primitives
|       |
|       |-- contexts/                # Global state
|       |   |-- AuthContext.jsx      # Authentication state
|       |   +-- LanguageContext.jsx   # i18n language selection
|       |
|       |-- i18n/                    # Translations
|       |   |-- en.js                # English
|       |   |-- es.js                # Spanish
|       |   +-- ar.js                # Arabic (RTL)
|       |
|       |-- data/
|       |   +-- mockData.js          # Development mock data
|       |
|       |-- utils/                   # Utility functions
|       |   +-- exportFile.js        # Excel/CSV download helper
|       |
|       +-- styles/
```

```
|           +-- index.css             # Global CSS + dark mode
|
|-- sap_mock/                         # === SAP MOCK SERVICE ===
|   |-- generate_mock_data.py         # Mock data generator
|   +-- data/                         # JSON datasets
|       |-- work_orders.json          # Historical work orders
|       |-- functional_locations.json # Equipment locations
|       |-- materials_bom.json        # Bill of materials
|       |-- equipment_master.json     # Equipment catalog
|       +-- maintenance_plans.json    # Predefined plans
|
|-- Dockerfile                        # Backend container
|-- docker-compose.yml                # 3-service orchestration
|-- nginx.conf                        # Reverse proxy configuration
|-- requirements.txt                  # Python dependencies (27 packages)
|-- .env.example                      # Environment variable template
+-- ocp_maintenance.db                # SQLite database (dev)
```

---

5. Backend (FastAPI)

5.1 Application Initialization

The application is created in `api/main.py` via the `create_app()` factory:

1. **FastAPI instance** with Swagger UI at `/docs`
2. **CORS middleware** — allows frontend origins from `.env`
3. **API Key middleware** — enforces `X-API-Key` on mutations when configured
4. **18 routers** registered under the `/api/v1` prefix
5. **Database tables** created at startup via the lifespan handler
6. **Health endpoint** at `/health` with DB connectivity check

5.2 Routers (18 Modules)

Router	Prefix	Purpose
`auth`	`/auth`	Login, registration, JWT tokens, user management
`hierarchy`	`/hierarchy`	CRUD for plants and equipment nodes
`criticality`	`/criticality`	Risk matrix assessment and approval
`fmea`	`/fmea`	Functions, failures, failure modes, RCM, FMECA
`tasks`	`/tasks`	Maintenance task CRUD
`work_packages`	`/work-packages`	Task grouping, approval, instructions
`sap`	`/sap`	SAP upload generation, mock data access
`analytics`	`/analytics`	Health scores, KPIs, Weibull analysis
`work_requests`	`/work-requests`	WR validation, duplicates, OCR
`capture`	`/capture`	Field data capture
`planner`	`/planner`	AI planner recommendations
`backlog`	`/backlog`	Backlog management and optimization
`scheduling`	`/scheduling`	Scheduling, GANTT
`reliability`	`/reliability`	Spare parts, shutdowns, MOCs
`reporting`	`/reporting`	Reports, notifications, export
`dashboard`	`/dashboard`	Executive KPIs and alerts
`rca`	`/rca`	Root Cause Analysis (5W2H)
`admin`	`/admin`	Seed, statistics, audit, feedback

5.3 Service Layer

Each router delegates business logic to a corresponding service in `api/services/`. Key services:

- **`auth_service.py`** — Password hashing (bcrypt), JWT creation/validation, user CRUD

- `fmea_service.py` — FMEA/FMECA logic, RCM decision trees, RPN calculation
- `analytics_service.py` — Weibull fitting, health score computation, KPI aggregation
- `criticality_service.py` — Risk matrix assessment, AI-suggested risk classes
- `work_package_service.py` — Task grouping algorithms, SAP upload package generation
- `reporting_service.py` — Weekly/monthly report generation, Excel export
- `rca_service.py` — 5W2H root cause analysis framework
- `capture_service.py` — Field capture processing with failure mode detection

5.4 Configuration

`api/config.py` loads configurations from environment variables:

```
DATABASE_URL        # SQLite or PostgreSQL connection string
SAP_MOCK_DIR        # Path to SAP mock data directory
JWT_SECRET_KEY      # Secret for signing JWT tokens
JWT_ALGORITHM       # HS256 (default)
CORS_ORIGINS        # Comma-separated allowed origins
API_V1_PREFIX       # /api/v1 (default)
LOG_LEVEL           # INFO, DEBUG, WARNING, etc.
```

---

6. Frontend (React + Vite)

6.1 Application Entry Point

`main.jsx` configures React Router with 23 lazy-loaded routes wrapped in role-based guards:

```
/login              -> Login.jsx         (public)
/                   -> Dashboard.jsx     (all roles)
/hierarchy          -> Hierarchy.jsx     (all roles)
/criticality        -> Criticality.jsx   (all roles)
/fmea               -> FMEA.jsx          (planner, engineer, admin)
/fmeca              -> FMECA.jsx         (planner, engineer, admin)
/strategy           -> Strategy.jsx      (planner, engineer, admin)
/work-requests      -> WorkRequests.jsx  (all roles)
/work-packages      -> WorkPackages.jsx  (planner, admin)
/backlog            -> Backlog.jsx       (planner, admin)
/scheduling         -> Scheduling.jsx    (planner, admin)
/planner            -> Planner.jsx       (planner, admin)
/planning           -> Planning.jsx      (planner, admin)
/field-capture      -> FieldCapture.jsx  (all roles)
/analytics          -> Analytics.jsx     (manager, admin)
/executive          -> ExecutiveDashboard (manager, admin)
/reports            -> Reports.jsx       (manager, admin)
/reliability        -> Reliability.jsx   (engineer, admin)
/rca                -> RCA.jsx           (engineer, admin)
/defect-elimination -> DefectElimination (engineer, admin)
/sap-review         -> SAPReview.jsx     (manager, admin)
/admin              -> Admin.jsx         (admin only)
/profile            -> Profile.jsx       (all roles)
```

6.2 Layout System

`App.jsx` provides the main layout: - **Sidebar** — Green navigation (#1B5E20) with OCP logo, role-filtered menu items, collapsible - **Header** — Plant selector, language selector (EN/ES/AR), dark mode, user menu - **Content Area** — Renders the active page via `<Outlet />`

6.3 State Management

Context	File	Purpose
`AuthContext`	`contexts/AuthContext.jsx`	User session, JWT tokens, login/logout, role checking
`LanguageContext`	`contexts/LanguageContext.jsx`	Active language, translator function `t()`, RTL detection

6.4 API Client

`api.js` provides 50+ functions wrapping HTTP calls to the backend:

- Base URL: `/api/v1` (Vite proxy in dev, Nginx in prod)
- Automatic `Authorization: Bearer {token}` header
- Automatic token refresh on 401 responses
- Centralized error handling

Example:

```
export const listWorkRequests = (params) => get('/work-requests', params);
export const validateWorkRequest = (id, d) => put(`/work-requests/${id}/validate`, d);
export const calculateHealthScore = (d) => post('/analytics/health-score', d);
```

6.5 UI Component Library

Built on **Radix UI** (headless, accessible) + **Tailwind CSS** (utility-first):

Component	Description
`Sidebar.jsx`	Main navigation with role-based filtering
`Header.jsx`	Top bar with plant/language/theme selectors
`ProtectedRoute.jsx`	Route guard that checks user roles
`Toast.jsx`	Success/error/info notification toasts
`ConfirmDialog.jsx`	Modal confirmation dialogs
`Shared.jsx`	`StatusBadge`, `LoadingSpinner`, `DataTable`
`ui/*.jsx`	15+ Radix primitives (Button, Dialog, Select, Tabs, etc.)

6.6 Theming

- **Light/dark mode** via custom CSS properties (`--background`, `--foreground`, etc.)
- **OCP brand color**: `#1B5E20` (dark green) as primary
- **Sidebar**: Solid green background `bg-[#1B5E20]`
- **Cards**: `bg-card border-border` adaptive to theme
- **Status badges**: Color-coded (green=approved, amber=draft, blue=review, red=critical)

---

7. Database Schema

7.1 Main Models (25+ Tables)

```
Users and Authentication
|-- UserModel              # Users with roles (admin, manager, planner, technician)

Asset Hierarchy
|-- PlantModel             # Manufacturing facilities
|-- HierarchyNodeModel     # Equipment tree (6 levels deep)

Criticality
|-- CriticalityAssessmentModel  # Risk matrix assessments with approval workflow

FMEA / FMECA
|-- FunctionModel              # Equipment functions
|-- FunctionalFailureModel     # Functional failures (TOTAL/PARTIAL)
|-- FailureModeModel           # Failure modes with mechanism/cause
|-- MaintenanceTaskModel       # RCM-derived tasks (CBM/TBM/FTM/RTF)

Work Management
|-- WorkPackageModel           # Grouped maintenance tasks
|-- WorkOrderModel             # Historical SAP work orders

Analytics
|-- HealthScoreModel           # Multidimensional asset health
|-- KPIMetricsModel            # MTBF, MTTR, availability, OEE
|-- FailurePredictionModel     # Weibull-based predictions

SAP Integration
|-- SAPUploadPackageModel      # SAP export packages

Continuous Improvement
|-- CAPAItemModel              # Corrective/preventive actions
```

7.2 Equipment Hierarchy Levels

```
PLANT (e.g., OCP-JFC1)
  +-- AREA (e.g., Grinding)
       +-- SYSTEM (e.g., SAG Mill Circuit)
            +-- EQUIPMENT (e.g., SAG Mill #1)
                 +-- SUB_ASSEMBLY (e.g., Drive Assembly)
                      +-- MAINTAINABLE_ITEM (e.g., Main Bearing)
```

---

8. Authentication and Authorization

8.1 JWT Token Flow

```
1. POST /auth/login { username, password }
   -> Returns: { access_token (4h), refresh_token (7d), user }

2. All API requests include:
   Authorization: Bearer <access_token>

3. On 401 (expired token):
   POST /auth/refresh { refresh_token }
   -> Returns: new access_token + refresh_token

4. If refresh fails:
   -> Redirect to /login
```

8.2 User Roles and Permissions

Role	Access
`admin`	Full system access + user management
`manager`	Operations oversight, analytics, reports, SAP review
`planner`	FMEA, work packages, backlog, scheduling, AI planner
`technician`	Dashboard, field capture, work requests, hierarchy
`engineer`	Reliability, RCA, defect elimination, FMEA

8.3 Default Demo Users

Username	Password	Role
`admin`	`admin123`	admin
`manager`	`manager123`	manager
`planner`	`planner123`	planner
`tecnico`	`tecnico123`	technician

8.4 Password Security

- Hashed with **bcrypt** (salt rounds)
- Never stored in plain text
- Minimum validation on registration

- Password change requires verification of previous password

---

9. API Reference

9.1 Base URL

```
Development:  http://localhost:8000/api/v1
Production:   https://your-domain.com/api/v1
Swagger UI:   http://localhost:8000/docs
```

9.2 Endpoint Summary

Authentication (`/auth`)

Method	Endpoint	Description
POST	`/auth/login`	Sign in with credentials
POST	`/auth/refresh`	Refresh JWT tokens
POST	`/auth/register`	Create new user (admin)
GET	`/auth/me`	Current user profile
PUT	`/auth/me`	Update profile
PUT	`/auth/change-password`	Change password
GET	`/auth/users`	List all users (admin)
PUT	`/auth/users/{id}/role`	Update user role
PUT	`/auth/users/{id}/activate`	Activate user
PUT	`/auth/users/{id}/deactivate`	Deactivate user

Hierarchy (`/hierarchy`)

Method	Endpoint	Description
GET	`/hierarchy/plants`	List plants
POST	`/hierarchy/plants`	Create plant
GET	`/hierarchy/nodes`	List equipment nodes
POST	`/hierarchy/nodes`	Create node
GET	`/hierarchy/nodes/{id}`	Get node details
GET	`/hierarchy/nodes/{id}/tree`	Get subtree
POST	`/hierarchy/build-from-vendor`	Import from vendor data
GET	`/hierarchy/stats`	Hierarchy statistics

FMEA (`/fmea`)

Method	Endpoint	Description
POST	`/fmea/functions`	Create equipment function
GET	`/fmea/functions`	List functions
POST	`/fmea/functional-failures`	Create functional failure
POST	`/fmea/failure-modes`	Create failure mode
GET	`/fmea/failure-modes`	List failure modes
GET	`/fmea/validate-fm`	Validate FM combination
GET	`/fmea/fm-combinations`	Valid mechanism/cause pairs
POST	`/fmea/rcm-decide`	RCM strategy decision
POST	`/fmea/fmeca/worksheets`	Create FMECA worksheet
POST	`/fmea/fmeca/rpn`	Calculate RPN

Analytics (`/analytics`)

Method	Endpoint	Description
POST	`/analytics/health-score`	Calculate asset health
POST	`/analytics/kpis`	Calculate KPI metrics
POST	`/analytics/weibull-fit`	Fit Weibull distribution
POST	`/analytics/weibull-predict`	Predict next failure
GET	`/analytics/variance-alerts`	Statistical anomalies
GET	`/analytics/asset-health`	Health summary

Reports (`/reporting`)

Method	Endpoint	Description
POST	`/reporting/reports/weekly`	Generate weekly report
POST	`/reporting/reports/monthly`	Generate monthly report
POST	`/reporting/reports/quarterly`	Generate quarterly report
GET	`/reporting/reports`	List reports
POST	`/reporting/export`	Export data (Excel/CSV/SAP)
GET	`/reporting/notifications`	List notifications
POST	`/reporting/cross-module/analyze`	Cross-module analysis

10. Internationalization (i18n)

10.1 Supported Languages

Language	Code	Direction	File
English	`en`	LTR	`frontend/src/i18n/en.js`
Spanish	`es`	LTR	`frontend/src/i18n/es.js`
Arabic	`ar`	RTL	`frontend/src/i18n/ar.js`

10.2 Implementation

The i18n system uses React Context (`LanguageContext.jsx`):

```
// Access translations in any component:
const { t, lang, setLang } = useLanguage();

// Usage:
t('nav.dashboard')                          // -> "Main Dashboard"
t('common.noResults', { query: 'pump' })    // -> "No results for 'pump'"
t('planner.suggestions')                    // -> ["Suggestion 1", "Suggestion 2", ...]
```

10.3 Translation Structure

```
{
  common: {        // Shared UI labels, buttons, states
  auth: {          // Login, roles, credentials
  nav: {           // Navigation menu items
  hierarchy: {     // Equipment hierarchy terms
  criticality: {   // Risk assessment terms
  fmea: {          // Failure mode analysis terms
  tasks: {         // Task management
  workPackages: {  // Work package operations
  analytics: {     // KPI labels and metrics
  reports: {       // Report types and export
  planner: {       // AI planner interface + responses
  strategy: {      // RCM strategy labels
  admin: {         // Admin panel
  // ... more modules
}
```

10.4 RTL Support

The Arabic layout automatically applies `dir="rtl"` to the root element, reversing flexbox, margins, padding, and text alignment.

11. SAP Integration

11.1 Overview

The platform generates SAP-compatible data packages for upload to the SAP PM (Plant Maintenance) module. During development, a SAP mock service provides realistic test data.

11.2 Mock Data (`sap_mock/data/`)

File	SAP Transaction	Content
`work_orders.json`	IW31/IW32	Historical PM orders
`functional_locations.json`	IL01	Technical locations
`materials_bom.json`	CS01	Bill of materials
`equipment_master.json`	IE01	Equipment master records
`maintenance_plans.json`	IP10	Predefined maintenance plans

11.3 SAP Upload Flow

```
1. Create Work Package (group tasks)
2. Approve Work Package
3. Generate SAP Upload Package
4. Review on SAP Review page
5. Approve for SAP transfer
6. Export in SAP-compatible format
```

11.4 SAP Field Mapping

```
OCP Field              -> SAP Field
work_order_id          -> AUFNR (Order Number)
equipment_tag          -> EQUNR (Equipment Number)
task_type (PM01-03)    -> AUART (Order Type)
priority               -> PRIOK (Priority)
description            -> KTEXT (Short Text)
planned_hours          -> ARBEI (Planned Work)
```

12. Docker and Deployment

12.1 Services (docker-compose.yml)

```
services:
  ocp-backend:      # FastAPI + Gunicorn (4 workers)
    ports: 8000
    volumes: ocp_db_data, ocp_sap_data
    healthcheck: HTTP GET /health

  ocp-frontend:     # React build served by Nginx
    ports: 5173
    depends_on: ocp-backend (healthy)

  ocp-nginx:        # Reverse proxy
    ports: 8080 (HTTP), 8443 (HTTPS)
    config: nginx.conf
```

12.2 Build Process

Backend (Dockerfile):

```
Stage 1: Python 3.11-slim -> Install dependencies
Stage 2: Copy packages -> Create non-root user -> Run Gunicorn
```

Frontend (frontend/Dockerfile):

```
Stage 1: Node 20-alpine -> npm ci -> npm run build
Stage 2: Nginx Alpine -> Copy dist/ -> SPA fallback routing
```

12.3 Docker Commands

```
# Build and start all services
docker-compose up -d --build

# View logs
docker-compose logs -f ocp-backend

# Stop all services
docker-compose down

# Rebuild a single service
docker-compose up -d --build ocp-frontend
```

12.4 Nginx Configuration

- **Rate limiting**: 30 req/s general, 2 req/s admin endpoints
- **Security headers**: X-Frame-Options, X-Content-Type-Options, X-XSS-Protection
- **Proxy pass**: `/api` and `/health` to backend, everything else to frontend

- **SSL ready**: Commented HTTPS configuration available for production

---

13. Functional Modules

Module 1: Asset Hierarchy and Criticality

Pages: Hierarchy, Criticality

- Build equipment tree from SAP master data or manual entry
- 6-level hierarchy: Plant -> Area -> System -> Equipment -> Sub-Assembly -> Maintainable Item
- Risk matrix assessment with AI-suggested risk classes
- Criteria: Safety, Environment, Production, Quality, Maintenance Cost
- Output: Risk classification (HIGH / MEDIUM / LOW)

Module 2: FMEA / FMECA and RCM Strategy

Pages: FMEA, FMECA, Strategy

- Define equipment functions and functional failures
- Create failure modes with standardized mechanisms and causes
- RPN (Risk Priority Number) calculation: Severity x Occurrence x Detection
- RCM Decision Logic:
- **CBM** — Condition-Based Maintenance (predictive)
- **TBM** — Time-Based Maintenance (preventive)
- **FTM** — Failure-Finding Task (hidden failures)
- **RTF** — Run-To-Failure (no action, accept risk)
- Generate maintenance tasks from RCM decisions

Module 3: Operations and Planning

Pages: WorkRequests, WorkPackages, Backlog, Scheduling, Planning, Planner, FieldCapture

- Field technicians capture data via mobile forms
- Work requests with AI-assisted priority classification
- Duplicate detection for similar requests
- Backlog management with aging analysis
- Task grouping into work packages by area/shutdown window
- Scheduling with GANTT visualization
- AI Planner chat interface with contextual recommendations

Module 4: Analytics, Reports, and Continuous Improvement

Pages: Analytics, ExecutiveDashboard, Reports, Reliability, RCA, DefectElimination, SAPReview

- **KPIs**: MTBF, MTTR, Availability, OEE, Schedule Adherence
- **Weibull Analysis**: Fit failure distributions, predict next failure
- **Health Scores**: Multidimensional asset health assessment
- **Reports**: Weekly, monthly, quarterly — exportable to Excel/CSV
- **RCA**: 5W2H framework (Who, What, When, Where, Why, How, How Much)
- **Defect Elimination**: Tracking recurring failures and corrective actions
- **SAP Review**: Audit generated upload packages before transfer

---

14. Key Workflows

14.1 Criticality Assessment

```
Select Equipment -> Fill Risk Criteria -> AI Suggests Risk Class
    -> User Approves/Modifies -> Save Assessment -> Feed into FMEA
```

14.2 FMEA to Work Package

```
Define Functions -> Identify Functional Failures -> Create Failure Modes
    -> Apply RCM Decision -> Generate Tasks -> Group into Work Packages
    -> Approve -> Export to SAP
```

14.3 Work Request Lifecycle

```
Field Capture -> Create Work Request -> Check for Duplicates
    -> AI Priority Classification -> Validate -> Add to Backlog
    -> Schedule -> Execute -> Close (OCR support)
```

14.4 Report Generation and Export

```
Select Report Type -> Configure Parameters -> Generate (API or Mock)
    -> Preview Results -> Export Excel/CSV -> Download
```

---

15. Security

Feature	Implementation
Authentication	JWT tokens (HS256), access 4 hours / refresh 7 days
Password Storage	bcrypt hash with salt
CORS	Origin whitelist from environment
API Key	Optional `X-API-Key` enforcement on mutations
Rate Limiting	Nginx: 30 req/s general, 2 req/s admin
Security Headers	X-Frame-Options: DENY, X-Content-Type-Options: nosniff
HTTPS	Nginx SSL/TLS 1.2+ ready (production configuration provided)
Role-Based Access	Frontend route guards + backend middleware
Non-Root Container	Backend runs as `appuser` (UID 1000)
SQL Injection Protection	SQLAlchemy ORM parameterized queries
CSRF Protection	JWT Bearer token (not cookie-based)

---

16. Local Development Guide

16.1 Prerequisites

- Python 3.11+
- Node.js 20+
- npm 9+
- Git

16.2 Backend Setup

```
# Clone and navigate
cd ASSET-MANAGEMENT-SOFTWARE-master

# Create virtual environment
python -m venv .venv
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate       # Windows

# Install dependencies
pip install -r requirements.txt

# Create .env (copy from example)
cp .env.example .env

# Start backend server
PYTHONPATH=. python -m uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
```

16.3 Frontend Setup

```
cd frontend

# Install dependencies
npm install

# Start development server
npm run dev
```

16.4 Access Points (Development)

URL	Service
`http://localhost:5173`	Frontend (React)
`http://localhost:8000`	Backend API
`http://localhost:8000/docs`	Swagger API Documentation
`http://localhost:8000/health`	Health Check

16.5 Seed Database

```
# Via API (requires running backend)
curl -X POST http://localhost:8000/api/v1/admin/seed

# Or via Python
PYTHONPATH=. python -c "from api.seed import seed_all; seed_all()"
```

---

17. Production Checklist

- [ ] Change `JWT_SECRET_KEY` to a strong random value
- [ ] Change `DATABASE_URL` to PostgreSQL
- [ ] Enable HTTPS in `nginx.conf` (uncomment SSL block)
- [ ] Generate SSL certificates (`certs/cert.pem` , `certs/key.pem`)
- [ ] Configure `API_KEY` and `ADMIN_API_KEY` for API protection
- [ ] Configure `CORS_ORIGINS` with production domains only
- [ ] Configure `ANTHROPIC_API_KEY` for AI features
- [ ] Set `DEBUG=false` and `LOG_LEVEL=WARNING`
- [ ] Review rate limiting in `nginx.conf`
- [ ] Configure volume mounts for persistent data
- [ ] Set up monitoring and log aggregation
- [ ] Run `docker-compose up -d --build`

---

18. Environment Variables

Variable	Default Value	Description
`DATABASE_URL`	`sqlite:///./ocp_maintenance.db`	Database connection string
`SAP_MOCK_DIR`	`sap_mock/data`	Path to SAP mock data
`JWT_SECRET_KEY`	`ocp-maintenance-secret-key-...`	JWT signing secret
`JWT_ALGORITHM`	`HS256`	JWT algorithm
`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`	`240`	Access token TTL
`JWT_REFRESH_TOKEN_EXPIRE_DAYS`	`7`	Refresh token TTL
`CORS_ORIGINS`	`http://localhost:5173,...`	Allowed CORS origins
`API_KEY`	*(empty)*	API key for mutation protection
`ADMIN_API_KEY`	*(empty)*	Admin API key
`ANTHROPIC_API_KEY`	*(empty)*	Claude AI API key
`GOOGLE_AI_API_KEY`	*(empty)*	Google AI API key
`DEBUG`	`false`	Enable debug logging
`LOG_LEVEL`	`INFO`	Python logging level
`BACKEND_PORT`	`8000`	Backend Docker port
`FRONTEND_PORT`	`5173`	Frontend Docker port
`NGINX_PORT`	`8080`	Nginx Docker HTTP port
`NGINX_SSL_PORT`	`8443`	Nginx Docker HTTPS port

Project Statistics

Metric	Count
API Routers	18
API Endpoints	100+
Database Models	25+
Backend Services	23
React Pages	23
UI Components	20+
Pydantic Schemas	50+
API Client Functions	50+
Supported Languages	3 (EN, ES, AR)
Docker Services	3
Python Dependencies	27
npm Dependencies	30+

> ***OCP Maintenance AI Platform*** *— Built for reliability, designed to scale.*